

```

// box_actions.h

#include <.common/comm.h>
#include <Arduino.h>
#include <SoftwareSerial.h>

#define MAX_BOXES 2
#define TIMEOUT_MS 500
#define TIME_TO_WAIT_FOR_RESPONSE 250

enum Pins
{
    FRESH_PIN = 2,
    GOING_BAD_PIN = 3,
    EXPIRED_PIN = 4,
    ALC_SENSOR_PIN = A0,
    CH3_SENSOR_PIN = A1
};

enum ThresholdPins // Current Testing Values
{
    ALC_THRESHOLD_BAD = 160,
    ALC_THRESHOLD_EXP = 500,
    CH3_THRESHOLD_BAD = 400,
    CH3_THRESHOLD_EXP = 600
};

FoodCondition evalLocal();
void RadioSetup();
void ParseBoxes(char *resp);

void Poller();

void NodeMcuUpdate(SoftwareSerial *sender);

// box_actions.cpp

#include "box_actions.h"

#include ".common/comm.h"

#include "interop_comms.h"
#include <Arduino.h>
#include <RF24.h>
#include <SoftwareSerial.h>
#include <stdint.h>

constexpr uint64_t MAIN_ADDRESS = 0xF0F0F0AA11LL;

RF24 radio(9, 10);

box_t active_boxes[MAX_BOXES];
uint8_t active_box_count = 0;

```

```
ThresholdValues thresholds;
```

```
inline void PrintAddress(uint64_t addr)
{
    d_SerialPrint(" 0x");

    for (int i = 4; i >= 0; i--) // nRF24 uses 5-byte addresses
    {
        uint8_t byte = (addr >> (8 * i)) & 0xFF;

        if (byte < 0x10)
            d_SerialPrintV('0');

        d_SerialPrintGroup(byte, HEX);
    }
    d_SerialPrintln("");
}
```

```
FoodCondition EvalLocal()
{
    int alc_reading = analogRead(Pins::ALC_SENSOR_PIN);
    int ch3_reading = analogRead(Pins::CH3_SENSOR_PIN);

    d_SerialPrint("Alcohol Values: ");
    d_SerialPrintlnV(alc_reading);
    d_SerialPrint("CH3 Values : ");
    d_SerialPrintlnV(ch3_reading);

    if (alc_reading >= thresholds.alc_exp || ch3_reading >= thresholds.ch3_exp)
    {
        d_SerialPrintln("\nExpired!\n\n");
        digitalWrite(Pins::EXPIRED_PIN, HIGH);
        digitalWrite(Pins::FRESH_PIN, LOW);
        digitalWrite(Pins::GOING_BAD_PIN, LOW);
        d_SerialPrint("Current Threshold Value is:\nAlc:");
        d_SerialPrintlnV(thresholds.alc_exp);
        d_SerialPrint("ch3:");
        d_SerialPrintlnV(thresholds.ch3_exp);
        return FoodCondition::EXPIRED;
    }
    else if (alc_reading >= thresholds.alc_bad || ch3_reading >= thresholds.ch3_bad)
    {
        d_SerialPrintln("\nGoing Bad!\n\n");
        digitalWrite(Pins::GOING_BAD_PIN, HIGH);
        digitalWrite(Pins::FRESH_PIN, LOW);
        digitalWrite(Pins::EXPIRED_PIN, LOW);
        d_SerialPrint("Current Threshold Value is:\nAlc:");
        d_SerialPrintlnV(thresholds.alc_bad);
        d_SerialPrint("ch3:");
        d_SerialPrintlnV(thresholds.ch3_bad);
        return FoodCondition::GOING_BAD;
    }
    else
```

```

{
    d_SerialPrintln("\nFresh!\n\n");
    digitalWrite(Pins::FRESH_PIN, HIGH);
    digitalWrite(Pins::GOING_BAD_PIN, LOW);
    digitalWrite(Pins::EXPIRED_PIN, LOW);
    return FoodCondition::FRESH;
}
}

void RadioSetup()
{
    radio.begin();
    radio.setPALevel(RF24_PA_LOW);
    radio.setDataRate(RF24_250KBPS);
    radio.setChannel(76);
    radio.setRetries(5, 15);

    radio.openReadingPipe(1, MAIN_ADDRESS);
    radio.startListening();

    d_SerialPrintGroup(radio.isChipConnected() ? "Radio initialized (UNO)" : " Radio
Uninitialized! Restart");
    d_SerialPrint("Listening on: ");
    PrintAddress(MAIN_ADDRESS);
    radio.stopListening();
}

void ParseBoxes(char *resp)
{
    memset(active_boxes, 0, sizeof(active_boxes));
    active_box_count = 0;

    char *token = strtok(resp, ";");

    while (token != NULL && active_box_count < MAX_BOXES)
    {
        while (*token == ' ')
            token++;

        size_t len = strlen(token);
        while (len > 0 && (token[len - 1] == ' ' || token[len - 1] == '\r' || token[len - 1]
== '\n'))
        {
            token[len - 1] = '\0';
            len--;
        }

        if (len >= 4) // Like, the input must atleast be something like "A0,1;", which is 4
[5]characters
        {
            char *comma = strchr(token, ',');
            if (comma != NULL)
            {

```

```

*comma = '\0';
char *valueStr = comma + 1;

uint8_t value = (uint8_t)atoi(valueStr);

// fill struct
active_boxes[active_box_count].index[0] = token[0];
active_boxes[active_box_count].index[1] = token[1];
active_boxes[active_box_count].index[2] = '\0';

// set state AFTER p_id is known
if (token[0] == 'A' && token[1] == '0')
{
    thresholds = profile_table[value];
    active_boxes[active_box_count].State = EvalLocal();
}
else
{
    active_boxes[active_box_count].State = FoodCondition::UNKNOWN;
    active_boxes[active_box_count].address =
CalcAddress(active_boxes[active_box_count].index);

    if (active_boxes[active_box_count].p_id != value) // Only update vals when
it's actually changed
    {
        active_boxes[active_box_count].p_id = (ProfileID)value;
        d_SerialPrint("Profile ID: ");
        d_SerialPrintlnV(active_boxes[active_box_count].p_id);

        Packet config;
        PackPacket(config, PacketType::PKT_CONFIG, 9, FoodCondition::UNKNOWN,
active_boxes[active_box_count].p_id);
        radio.openWritingPipe(active_boxes[active_box_count].address);

        radio.stopListening();

        radio.openWritingPipe(active_boxes[active_box_count].address);
        bool ok = radio.write(&config, sizeof(config));

        if (!ok)
            d_SerialPrintln("Config upload failed");
    }
}

active_box_count++;
}
}

token = strtok(NULL, ";");
}

d_SerialPrint("Parsed ");
d_SerialPrintV(active_box_count);

```

```

d_SerialPrintln(" boxes");

for (int i = 0; i < active_box_count; i++)
{
    d_SerialPrint("Box ");
    d_SerialPrintV(i);
    d_SerialPrint(": ");
    d_SerialPrintlnV(active_boxes[i].index);
}
}

void ClearRadioRX()
{
    while (radio.available())
    {
        Packet junk;
        radio.read(&junk, sizeof(junk));
    }
}

void Poller()
{
    static int current_box = 0;
    static uint8_t seq = 0;

    if (active_box_count == 0)
        return;

    if (current_box >= active_box_count)
        current_box = 0;

    int start_box = current_box;

    while (strcmp(active_boxes[current_box].index, "A0") == 0)
    {
        current_box++;

        if (current_box >= active_box_count)
            current_box = 0;

        if (current_box == start_box) // If only A0 exists
            return;
    }

    uint8_t my_seq = seq;

    Packet req;
    PackPacket(req, PacketType::PKT_REQUEST, my_seq, FoodCondition::UNKNOWN,
active_boxes[current_box].p_id);

    d_SerialPrint("Polling box: ");
    d_SerialPrintlnV(active_boxes[current_box].index);
}

```

```

d_SerialPrint("Polling address: ");
PrintAddress(active_boxes[current_box].address);

ClearRadioRX();
radio.stopListening();
radio.openWritingPipe(active_boxes[current_box].address);

bool ok = radio.write(&req, sizeof(req));

d_SerialPrint("Poller write = ");
d_SerialPrintlnV(ok ? "OK" : "FAIL");

radio.openReadingPipe(1, MAIN_ADDRESS);
radio.startListening();

unsigned long start = millis();
bool got_matching_response = false;

while (millis() - start < TIME_TO_WAIT_FOR_RESPONSE)
{
    if (!radio.available())
        continue;

    Packet resp;
    radio.read(&resp, sizeof(resp));

    uint8_t type, rseq;
    FoodCondition cond;
    uint8_t flags;

    if (!UnpackPacket(resp, type, rseq, cond, flags))
    {
        d_SerialPrintln("Poller: bad response packet");
        continue;
    }

    if (type != PKT_RESPONSE)
    {
        d_SerialPrintln("Silly Nano! You have sent the wrong type of request.");
        continue;
    }

    if (rseq != my_seq)
    {
        d_SerialPrint("Poller: seq mismatch, got ");
        d_SerialPrintV(rseq);
        d_SerialPrint(" expected ");
        d_SerialPrintlnV(my_seq);
        continue;
    }

    d_SerialPrintln("Poller: valid response");
    active_boxes[current_box].State = cond;
}

```



```

    got_matching_response = true;
    break;
}

if (!got_matching_response)
{
    d_SerialPrintln("Poller: response timeout");
    active_boxes[current_box].State = FoodCondition::UNKNOWN;
}

radio.stopListening();
radio.openReadingPipe(1, MAIN_ADDRESS);
radio.startListening();

current_box++;

if (current_box >= active_box_count)
    current_box = 0;

seq = (seq + 1) & 0x0F;
}

void NodeMcuUpdate(SoftwareSerial *sender)
{
    char csv_data[120];
    size_t idx = 0;

    for (int i = 0; i < active_box_count; i++)
    {
        const char *index = active_boxes[i].index;
        uint8_t state = (uint8_t)active_boxes[i].State;

        int written = snprintf(csv_data + idx, sizeof(csv_data) - idx, "%s,%d\n", index,
state);

        if (written < 0)
            break;

        idx += (size_t)written;

        if (idx >= sizeof(csv_data))
        {
            idx = sizeof(csv_data) - 1;
            break;
        }
    }

    csv_data[idx] = '\0';

    char dummy[16] = {0};

    if (!NodeMCUTransmit(Requests::UpdateBoxes, sender, csv_data, dummy, sizeof(dummy)))

```

```
d_SerialPrintln("Update failed");  
}
```